Guía de estudio
TypeScript, JavaScript, APIs y React

Cada tema explicado desde cero, con ejemplos de código,
errores comunes y puntos clave para entrevistas
(con notas específicas para React Native)

Cómo usar esta guía

Esta guía toma tu temario y explica cada punto con tres ingredientes: **qué es**, **un ejemplo** y **qué se suele preguntar**. Los recuadros amarillos marcan notas de React Native o consejos para la entrevista.

Una aclaración: el temario que pegaste mezcla TypeScript, JavaScript, programación funcional, backend (HTTP, GraphQL, ORMs) y React. Todo eso es relevante para un puesto senior de React Native, porque normalmente tendrás que dialogar con APIs y backends, y TypeScript es la base del código. Los temas puramente web (por ejemplo, el DOM) los explico y luego señalo su equivalente en React Native.

Orden de estudio recomendado

Semana	Bloque	Objetivo
1	Parte 1: TypeScript (temas 1 a 7)	Dominar el sistema de tipos: interfaces, uniones, genéricos, narrowing, utility types.
2	Parte 1 (8 a 13) + Parte 2: paradigmas	Configuración, validación en runtime, estructuras de datos, complejidad, programación funcional.
3	Partes 3 y 4: librerías, HTTP, GraphQL, auth, asincronía	Entender el lado servidor y cómo consumirlo bien desde la app.
4	Partes 5 y 6: bases de datos y React	Rendimiento de consultas, hooks, efectos, formularios, accesibilidad. Repaso general.

Método sugerido: lee un tema, cierra la guía y escribe el ejemplo de memoria en el TypeScript Playground (typescriptlang.org/play). Si puedes explicarlo con tus palabras, ya lo dominas.

Parte 1. TypeScript y fundamentos de JavaScript

1. Interfaces y type aliases

TypeScript usa **tipado estructural**: dos tipos son compatibles si tienen la misma forma, sin importar cómo se llamen. Si un objeto tiene las propiedades que se piden, sirve (a diferencia de Java o C#, donde importa el nombre de la clase).

Hay dos formas de describir objetos: **interface** y **type alias**. En la mayoría de los casos son intercambiables.

- **Extender:** una interface usa `extends`; un type usa intersección `&`. Ambos permiten componer tipos pequeños en otros mayores.
- **Declaration merging:** si declaras dos veces la misma `interface`, TypeScript las fusiona. Un `type` repetido da error. Esto es útil para ampliar tipos de librerías (por ejemplo, añadir campos a `Window` o a los params de React Navigation).
- **type alias** puede representar cosas que una interface no puede: uniones (`string | number`), tuplas, tipos primitivos, tipos mapeados y condicionales.
- **Cuál elegir:** `interface` para formas de objeto públicas y extensibles (props de componentes, contratos de clases); `type` para uniones, utilidades y tipos calculados. Lo importante es ser consistente con la convención del proyecto.

```
interface User { id: string; name: string }
interface Admin extends User { permissions: string[] }

type Id = string | number;                        // solo un type puede
type Timestamps = { createdAt: Date; updatedAt: Date };
type Post = { title: string } & Timestamps;       // composicion con &

// Tipado estructural: no hace falta "implements"
const u = { id: '1', name: 'Ana', extra: true };
function greet(user: User) { return user.name; }
greet(u);                                         // OK: tiene id y name

// Declaration merging
interface Config { apiUrl: string }
interface Config { timeout: number }              // Config ahora tiene ambas
```

Para la entrevista: explica que la diferencia práctica más citada es el declaration merging y que `type` admite uniones. Mostrar que conoces ambos y que eliges por caso de uso gana puntos.

2. Enums y tipos literales

Un **tipo literal** es un tipo con un valor exacto: `'success'` es un tipo, no solo un string. Con una unión de literales (`'success' | 'error'`) restringes una variable a unos pocos valores válidos.

Un **enum** es una construcción propia de TypeScript que crea un objeto real en tiempo de ejecución.

- **Enum de strings:** `enum Status { Success = 'success' }`. Es legible al depurar.
- **Enum numérico:** `enum Dir { Up, Down }` asigna 0, 1... y permite mapeo inverso (`Dir[0]`), pero es más propenso a errores y menos legible en logs.
- **Unión de literales:** no genera código JavaScript (se borra al compilar), es simple y se integra bien con inferencia y narrowing.
- **Objeto** `as const`**:** te da valores en runtime (para iterar o comparar) y, derivando el tipo, una unión de literales. Es la alternativa moderna más usada.
- **Tradeoffs de enum:** emiten código extra, no son 'borrables' (chocan con la opción `erasableSyntaxOnly` y con algunos transpiladores), y `const enum` da problemas con

`isolatedModules` (Babel/Metro). Por eso muchos equipos prefieren uniones o `as const`.

```
type Status = 'success' | 'error';              // union de literales

const STATUS = { Success: 'success', Error: 'error' } as const;
type StatusValue = typeof STATUS[keyof typeof STATUS];  // 'success' | 'error'

function show(s: StatusValue) { /* ... */ }
show(STATUS.Success);
show('error');                                   // OK
// show('loading');                              // Error de compilacion
```

En React Native: Metro compila TypeScript con Babel archivo por archivo, y eso hace que `const enum` y algunos usos de `enum` sean problemáticos. Una unión de literales o un objeto `as const` es la opción más segura.

3. Inferencia de tipos y aserciones

TypeScript **infiere** tipos a partir de los valores, por lo que no necesitas anotar todo. `const n = 5` es de tipo `5` (literal) y `let n = 5` de tipo `number`. El **tipado contextual** infiere el tipo de un parámetro según dónde se usa (por ejemplo, el `e` de un callback de `onPress`).

Una **aserción** (`as Tipo`) le dice al compilador 'confía en mí, es de este tipo'. Solo cambia lo que TypeScript cree; **no valida nada en tiempo de ejecución**. Si te equivocas, el error aparecerá en producción.

- `satisfies` (TS 4.9): comprueba que un valor cumple un tipo *sin ampliar ni perder* su tipo inferido más preciso. Ideal para configuraciones y objetos de rutas.
- `as const`**:** congela el valor a su forma literal y de solo lectura.
- **Non-null** `!`**:** afirma que algo no es null/undefined. Úsalo con moderación.
- **Doble aserción** `as unknown as T`**:** señal de alerta; fuerza un tipo y casi siempre oculta un problema.

```
const palette = {
  red: [255, 0, 0],
  green: '#00ff00',
} satisfies Record<string, string | number[]>;

palette.red.map(x => x * 2);   // OK: sigue siendo number[]

// Aserciones: no validan
const data = JSON.parse(text) as User;   // si el JSON no es un User, TS no avisa
data.name.toUpperCase();                 // puede explotar en runtime
```

Para la entrevista: la frase clave es 'las aserciones no hacen validación en runtime'; la solución son guards o librerías como Zod (tema 10).

4. Funciones y genéricos

Tipar funciones es anotar **parámetros y retorno**. Aunque el retorno suele inferirse, anotarlo en funciones exportadas documenta el contrato y evita cambios accidentales.

Los **genéricos** permiten escribir código reutilizable que conserva el tipo: en lugar de `any`, usas un parámetro de tipo `T` que se resuelve según el uso.

- **Sobrecargas (overloads):** varias firmas para una misma función, con una sola implementación. Útil cuando el tipo de retorno depende del de entrada.
- **Genéricos en funciones, interfaces y clases:** `function first<T>(a: T[])`, `interface Result<T>`, `class Stack<T>`.
- **Constraints (restricciones):** `<T extends { length: number }>` exige que `T` tenga cierta forma, para poder usarla dentro de la función.

- **Valores por defecto:** `<T = string>` y múltiples parámetros `<K extends keyof T, T>`.

```
// Sobrecargas
function parse(x: string): number;
function parse(x: number): string;
function parse(x: string | number) {
  return typeof x === 'string' ? Number(x) : String(x);
}

// Generico simple
function first<T>(arr: T[]): T | undefined { return arr[0]; }
const n = first([1, 2, 3]);      // number | undefined

// Constraint
function longest<T extends { length: number }>(a: T, b: T): T {
  return a.length >= b.length ? a : b;
}
longest('hola', 'adios');        // OK
longest([1, 2], [3]);            // OK
// longest(10, 20);              // Error: number no tiene length

// Interface y clase genericas
interface Result<T> { ok: boolean; data?: T; error?: string }
class Stack<T> {
  private items: T[] = [];
  push(x: T) { this.items.push(x); }
  pop(): T | undefined { return this.items.pop(); }
}
```

En React Native: el hook `useState<User | null>(null)` y `useRef<TextInput>(null)` son genéricos que usarás a diario.

5. Tipos unión e intersección

Unión (`|`): un valor puede ser *uno u otro* tipo. Solo puedes usar con seguridad lo que es común a todos, hasta que estreches el tipo (narrowing).

Intersección (`&`): el valor debe cumplir *ambos* tipos a la vez; sirve para combinar formas de objeto compatibles.

- Si intersectas tipos incompatibles (`string & number`) obtienes `never`, un tipo sin valores posibles.
- Las uniones modelan *estados alternativos* (cargando | éxito | error); las intersecciones modelan *composición* (usuario con timestamps).

```
type Id = string | number;
function print(id: Id) {
  // id.toUpperCase();   // Error: number no tiene toUpperCase
  if (typeof id === 'string') id.toUpperCase();   // OK, estrechado
}

type Named = { name: string };
type Aged = { age: number };
type Person = Named & Aged;              // { name: string; age: number }
const p: Person = { name: 'Ana', age: 30 };

type Impossible = string & number;       // never
```

6. Narrowing (estrechamiento) y type guards

Narrowing es cuando TypeScript reduce un tipo amplio a uno más específico dentro de un bloque, según comprobaciones que haces en el código. Es lo que hace útiles las uniones.

- `typeof`**:** para primitivos (`'string'`, `'number'`, `'boolean'`...).
- `instanceof`**:** para clases (`error instanceof Error`).

- `in`: comprueba si existe una propiedad (`'radius' in shape`).
- **Igualdad:** `x === null`, `x === 'error'`, y la comprobación de truthiness.
- **Type predicate definido por ti:** una función con retorno `value is Tipo` que encapsula una comprobación reutilizable.
- **Uniones discriminadas:** objetos con una propiedad común (`kind`, `status`, `type`) con valor literal distinto en cada variante. Es el patrón más potente y el favorito de las entrevistas.
- **Comprobación exhaustiva con `never`:** si añades una variante nueva y olvidas manejarla, el compilador avisa.

```
// Predicado de tipo
function isString(v: unknown): v is string { return typeof v === 'string'; }

// Union discriminada
type State =
  | { status: 'loading' }
  | { status: 'success'; data: string[] }
  | { status: 'error'; message: string };

function render(s: State) {
  switch (s.status) {
    case 'loading': return 'Cargando...';
    case 'success': return s.data.join(', ');     // aqui TS sabe que existe data
    case 'error':   return s.message;
    default: {
      const _exhaustive: never = s;               // error si falta una variante
      return _exhaustive;
    }
  }
}

// catch con unknown
try { risky(); } catch (e) {
  if (e instanceof Error) console.log(e.message);
}
```

En React Native: modela el estado de una pantalla (loading / success / error) con una unión discriminada en vez de tres booleanos (`isLoading`, `isError`, `data`); evita estados imposibles.

7. Utility types

Son tipos genéricos que vienen con TypeScript para **transformar tipos existentes** sin reescribirlos.

Utility	Qué hace	Ejemplo
`Partial<T>`	Todas las propiedades pasan a opcionales	`Partial<User>` para un formulario de edición
`Required<T>`	Todas pasan a obligatorias	Config final tras aplicar defaults
`Pick<T, K>`	Toma solo algunas propiedades	`Pick<User, 'id' | 'name'>`
`Omit<T, K>`	Excluye algunas propiedades	`Omit<User, 'password'>`
`Record<K, V>`	Objeto con claves K y valores V	`Record<string, number>`
`Readonly<T>`	Propiedades de solo lectura	Estado inmutable
`ReturnType<F>`	Tipo que devuelve una función	`ReturnType<typeof useAuth>`
`Parameters<F>`	Tupla con los parámetros de una función	Envolver/mockear funciones
`Awaited<T>`	Tipo resuelto de una Promise	`Awaited<ReturnType<typeof fetchUser>>`

```
interface User { id: string; name: string; email: string; password: string }
```

```
type PublicUser = Omit<User, 'password'>;
type UserPatch  = Partial<Omit<User, 'id'>>;
type Roles      = Record<'admin' | 'user', string[]>;
type LoginFn    = (email: string, pass: string) => Promise<User>;
type LoginArgs  = Parameters<LoginFn>;          // [email: string, pass: string]

// Mapped type propio
type Nullable<T> = { [K in keyof T]: T[K] | null };

// Conditional type propio
type ElementType<T> = T extends (infer U)[] ? U : T;
type A = ElementType<string[]>;                 // string
```

¿Cuándo crear tipos mapeados o condicionales propios? Cuando un patrón se repite en varios lugares y los utility types no lo cubren, o al construir librerías/utilidades genéricas. Para código de aplicación normal, suelen ser un exceso de ingeniería: tipos muy complejos se vuelven difíciles de leer y ralentizan el compilador.

8. APIs del DOM

El **DOM** es el árbol de elementos de una página web. TypeScript incluye tipos para él (`HTMLElement`, `HTMLInputElement`, `MouseEvent`, `KeyboardEvent`...). Aplica a web, no a React Native, pero los conceptos de fondo sí se reutilizan.

- `querySelector` **puede devolver** `null`**:** debes comprobarlo antes de usarlo (narrowing).
- **El genérico no verifica:** `querySelector<HTMLInputElement>('#email')` es una aserción disfrazada; si el elemento no es un input, TypeScript no lo sabe. Para seguridad real usa `instanceof HTMLInputElement`.
- **Brecha compile-time vs runtime:** los tipos del DOM describen lo que *esperas*, pero el HTML real puede ser distinto (id faltante, otro tag, cargado después). El código debe tolerarlo.
- **Eventos:** `event.target` es `EventTarget | null` (poco específico); `event.currentTarget` está mejor tipado.

```
const el = document.querySelector('#email');
if (!(el instanceof HTMLInputElement)) {
  throw new Error('No se encontro el input #email');
}
el.value = 'ana@correo.com';         // aqui TS sabe que es HTMLInputElement

el.addEventListener('input', (e: Event) => {
  const value = (e.currentTarget as HTMLInputElement).value;
});
```

En React Native: no hay DOM. El equivalente son las **refs**: `const ref = useRef<TextInput>(null);` `ref.current` puede ser `null`, así que se comprueba con `ref.current?.focus()`. Los eventos tienen sus propios tipos, como `NativeSyntheticEvent<TextInputChangeEventData>`.

9. Configuración de proyectos TypeScript

TypeScript se configura con `tsconfig.json`. Lo más importante es activar el **modo estricto**, que habilita un conjunto de comprobaciones que evitan la mayoría de errores.

- `strict: true`**:** incluye `strictNullChecks`, `noImplicitAny`, `strictFunctionTypes`, etc. En proyectos nuevos, actívalo siempre.
- **Opciones extra útiles:** `noUncheckedIndexedAccess` (acceder a `arr[i]` da `T | undefined`), `noImplicitOverride`, `exactOptionalPropertyTypes`.
- `target`**:** versión de JavaScript que se emite. `module` **y** `moduleResolution`**:** cómo se resuelven los imports (`bundler` es el valor habitual con Metro/Vite/webpack).

- **Archivos de declaración (`.d.ts`):** describen los tipos de código JavaScript sin implementarlo. Se usan para librerías sin tipos (`@types/...`) o para declarar módulos globales (por ejemplo, imágenes `.png`).
- **Migración incremental JS a TS:** activar `allowJs`, opcionalmente `checkJs` o comentarios `// @ts-check` por archivo; renombrar archivo por archivo a `.ts/.tsx`; empezar con `strict` flexible y endurecer gradualmente; tipar primero los límites (API, modelos) y las utilidades compartidas.

```
{
  "compilerOptions": {
    "strict": true,
    "target": "ES2020",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "jsx": "react-native",
    "noUncheckedIndexedAccess": true,
    "allowJs": true,
    "skipLibCheck": true,
    "noEmit": true
  },
  "include": ["src"]
}

// declaraciones.d.ts
declare module '*.png' { const src: number; export default src; }
```

En React Native: Metro/Babel **no comprueba tipos**, solo los elimina. Debes correr `tsc --noEmit` en CI y en un script de pre-commit; si no, los errores de tipos nunca bloquean el build. Expo y React Native proveen una base (`extends`) lista para usar.

10. Validación en runtime

Los tipos de TypeScript **desaparecen al compilar**. Todo lo que llega desde fuera de tu código (respuestas de API, `AsyncStorage`, deep links, parámetros de URL, formularios) puede tener cualquier forma, aunque tu tipo diga otra cosa.

- Se resuelve validando en el borde con librerías como **Zod**, Valibot o io-ts, y deduciendo el tipo estático del esquema (una sola fuente de verdad).
- Beneficios: detectas cambios inesperados del backend, evitas crashes por `undefined` y puedes transformar datos (fechas, valores por defecto).
- Usa `safeParse` si quieres manejar el error sin lanzar excepción.

```
import { z } from 'zod';

const UserSchema = z.object({
  id: z.string(),
  email: z.string().email(),
  age: z.number().int().min(0).optional(),
});
type User = z.infer<typeof UserSchema>;

const json: unknown = await res.json();
const result = UserSchema.safeParse(json);
if (!result.success) {
  logError(result.error);           // datos invalidos
} else {
  const user: User = result.data;   // ahora si es seguro
}
```

Para la entrevista: 'un `as User` no valida nada; los tipos se borran en runtime, por eso valido en el borde con Zod'.

11. Manejo de errores, inmutabilidad y funciones pequeñas

Un código robusto y fácil de mantener combina tres hábitos.

- **Manejo de errores:** usa `try/catch` en los límites (llamadas de red, parseo), crea errores propios (`class ApiError extends Error`), trata el `catch` como `unknown` y no tragues errores en silencio. Una alternativa funcional es devolver un **Result** (`{ ok: true, value } | { ok: false, error }`) en lugar de lanzar excepciones, haciendo visibles los fallos en los tipos.
- **Inmutabilidad:** no modificar datos existentes; crear copias con spread (`{ ...obj, a: 1 }`, `[...arr, x]`), `map`/`filter` o `structuredClone`. React y Redux dependen de ello para detectar cambios. `readonly` y `as const` lo refuerzan en tipos.
- **Funciones pequeñas y componibles:** cada función hace una cosa, es fácil de probar, y se combinan entre sí (composición).

```
class ApiError extends Error {
  constructor(public status: number, message: string) { super(message); }
}

type Result<T> = { ok: true; value: T } | { ok: false; error: string };

// Inmutabilidad
const state = { user: { name: 'Ana' }, count: 0 };
const next  = { ...state, count: state.count + 1 };       // state no cambia
const list  = [1, 2, 3];
const added = [...list, 4];
const removed = list.filter(x => x !== 2);

// Composicion
const pipe = <T,>(...fns: Array<(x: T) => T>) => (x: T) => fns.reduce((v, f) => f(v), x);
const slugify = pipe<string>(
  s => s.trim(),
  s => s.toLowerCase(),
  s => s.replace(/\s+/g, '-'),
);
```

12. Estructuras de datos en JavaScript

Elegir la estructura adecuada depende de **cómo accederás a los datos**: ¿buscar por clave?, ¿mantener orden?, ¿evitar duplicados?, ¿recorrer relaciones?

Estructura	Úsala cuando...	Costos típicos
Array	Necesitas orden y acceso por posición	Acceso por índice O(1); buscar `includes` O(n); `shift/unshift` O(n)
Objeto	Datos con claves conocidas (registros)	Acceso por clave O(1) promedio
`Map`	Pares clave-valor con claves de cualquier tipo, o muchas altas/bajas	get/set/delete O(1); mantiene orden de inserción
`Set`	Eliminar duplicados o comprobar pertenencia rápido	add/has/delete O(1)
Pila (stack)	Último en entrar, primero en salir: deshacer, recursión, paréntesis	`push`/`pop` O(1)
Cola (queue)	Primero en entrar, primero en salir: tareas, BFS	`push` O(1); `shift` en array es O(n), mejor lista enlazada o índice
Árbol	Jerarquías: carpetas, menús, árbol de componentes, búsqueda ordenada	Recorrido O(n); BST balanceado O(log n)
Grafo	Relaciones: redes sociales, rutas, dependencias	Lista de adyacencia: recorrer O(V + E)

```
// Map para contar frecuencias
```

```
const count = new Map<string, number>();
for (const w of words) count.set(w, (count.get(w) ?? 0) + 1);

// Set para quitar duplicados
const unique = [...new Set([1, 2, 2, 3])];

// Pila
const stack: number[] = [];  stack.push(1);  stack.pop();

// Arbol
type TreeNode = { value: number; children: TreeNode[] };

// Grafo (lista de adyacencia) y recorrido BFS
const graph = new Map<string, string[]>([
  ['A', ['B', 'C']], ['B', ['D']], ['C', ['D']], ['D', []],
]);
function bfs(start: string) {
  const seen = new Set([start]);
  const queue = [start];
  for (let i = 0; i < queue.length; i++) {     // indice en vez de shift()
    for (const nb of graph.get(queue[i]) ?? []) {
      if (!seen.has(nb)) { seen.add(nb); queue.push(nb); }
    }
  }
  return [...seen];
}
```

Para la entrevista: cuando veas un problema, pregúntate 'qué operación repito más' y elige la estructura que la haga barata. Normalizar datos en un `Map` por `id` es un truco clásico para evitar búsquedas lineales.

13. Análisis de complejidad

La **notación Big-O** describe cómo crece el costo (tiempo o memoria) al aumentar el tamaño de la entrada `n`. No mide segundos, mide *crecimiento*.

Complejidad	Nombre	Ejemplo común
O(1)	Constante	Acceso por índice, `Map.get`, `Set.has` (promedio)
O(log n)	Logarítmica	Búsqueda binaria
O(n)	Lineal	Recorrer un array, `find`, `includes`, `map`, `filter`
O(n log n)	Linealítmica	Ordenar (`sort`)
O(n^2)	Cuadrática	Dos bucles anidados sobre la misma lista
O(2^n)	Exponencial	Fibonacci recursivo ingenuo

```
// Dos sumas: encontrar dos numeros que sumen target
// O(n^2): bucle anidado
function twoSumSlow(a: number[], t: number) {
  for (let i = 0; i < a.length; i++)
    for (let j = i + 1; j < a.length; j++)
      if (a[i] + a[j] === t) return [i, j];
}

// O(n) tiempo, O(n) memoria: busqueda hash
function twoSum(a: number[], t: number) {
  const seen = new Map<number, number>();          // valor -> indice
  for (let i = 0; i < a.length; i++) {
    const j = seen.get(t - a[i]);
    if (j !== undefined) return [j, i];
    seen.set(a[i], i);
  }
}
```

- **Tiempo vs espacio:** a menudo se cambia memoria por velocidad (el `Map` de arriba usa más memoria pero reduce el tiempo).
- **Ordenar** cuesta O(n log n); si ya está ordenado, puedes usar búsqueda binaria O(log n).
- **Ojo con los métodos ocultos:** `includes` dentro de un `filter` o un `map` es un O(n^2) disfrazado. Convierte a `Set` primero.
- **En listas de React Native** un cálculo O(n^2) dentro de `renderItem` se nota enseguida en el rendimiento.

Parte 2. Paradigmas de programación y programación funcional

Imperativo vs declarativo

Imperativo: describes *cómo* hacerlo, paso a paso (bucles, variables que cambian). **Declarativo:** describes *qué* quieres, y otra capa decide cómo (SQL, CSS, JSX, `map`/`filter`). React es declarativo: describes la UI para un estado y React se encarga de actualizarla.

```
// Imperativo
const result = [];
for (let i = 0; i < nums.length; i++) {
  if (nums[i] % 2 === 0) result.push(nums[i] * 2);
}

// Declarativo
const result2 = nums.filter(n => n % 2 === 0).map(n => n * 2);
```

Estilos: orientado a objetos, funcional y orientado a eventos

- **Orientado a objetos (POO):** modelas con clases que agrupan datos y comportamiento (encapsulación, herencia, polimorfismo). Útil para entidades con identidad y estado propio.
- **Funcional:** funciones puras, datos inmutables y composición. Fácil de probar y de razonar.
- **Orientado a eventos:** el flujo lo marcan eventos y callbacks (toques de usuario, notificaciones push, WebSockets, `EventEmitter`). Es la naturaleza de las apps móviles.

Elegir según el problema

No hay un paradigma 'mejor'. En la práctica se mezclan: un componente React funcional (FP) que responde a eventos (event-driven) y usa un servicio con clase (POO) para hablar con la API. El criterio senior es **elegir lo que haga el código más claro para el problema**, no aplicar un patrón en todas partes.

Conceptos de programación funcional

- **Funciones puras:** mismo input produce siempre el mismo output y no tienen efectos secundarios. Son fáciles de probar y de memoizar.
- **Inmutabilidad:** en lugar de cambiar un valor, devuelves uno nuevo.
- **Composición:** combinar funciones pequeñas en otras mayores (`pipe`, `compose`).
- **Funciones de orden superior:** reciben funciones como argumento o las devuelven (`map`, `filter`, `useCallback`, un HOC).
- **Evitar efectos ocultos:** una función que modifica una variable global, lee la hora o hace una petición sin que se note en su firma es difícil de probar y depurar.

```
// Impura: depende de estado externo y lo modifica
let total = 0;
function add(x: number) { total += x; return total; }

// Pura
const add2 = (total: number, x: number) => total + x;

// Orden superior
const withLogging = <A extends unknown[], R>(fn: (...a: A) => R) =>
  (...args: A): R => { console.log('llamada', args); return fn(...args); };
```

map, filter, reduce, find y cuándo un bucle es más claro

- `map`: transforma cada elemento (mismo largo). `filter`: se queda con los que cumplen. `find`: primer elemento que cumple (o `undefined`). `some`/`every`: comprueban condiciones. `reduce`: acumula en un solo valor. `flatMap`: map + aplanar.
- `reduce` **se abusa:** si el resultado es un objeto complejo o hay mucha lógica, un `for...of` suele ser más legible. Si tienes que leerlo dos veces, usa un bucle.
- **Un bucle gana** cuando necesitas `break`/`continue`, manejo de varios índices, `await` secuencial o rendimiento crítico (evitar encadenar varios `map`/`filter` sobre listas enormes).

```
const users = [{ name: 'Ana', age: 30 }, { name: 'Luis', age: 17 }];

const adultNames = users.filter(u => u.age >= 18).map(u => u.name);
const first = users.find(u => u.name === 'Luis');
const totalAge = users.reduce((sum, u) => sum + u.age, 0);

// Agrupar con reduce (valido) o con un bucle (a veces mas claro)
const byAge: Record<number, string[]> = {};
for (const u of users) (byAge[u.age] ??= []).push(u.name);
```

Efectos secundarios en los límites

Un **efecto secundario** es cualquier cosa que afecta al mundo exterior: llamadas de red, almacenamiento, logs, actualizar la UI. No se pueden eliminar, pero sí **aislarlos en los bordes**: el núcleo de tu app (lógica de negocio) se mantiene puro y fácil de probar, y una capa fina se encarga de las entradas y salidas. Se le llama a veces 'núcleo funcional, cáscara imperativa'.

```
// Nucleo puro: facil de testear
export function calculateDiscount(total: number, isVip: boolean) {
  return isVip ? total * 0.9 : total;
}

// Borde con efectos: llama a la red y guarda
async function checkout(cart: Cart) {
  const total = calculateDiscount(cart.total, cart.user.isVip);   // puro
  const res = await api.post('/orders', { total });               // efecto
  await storage.set('lastOrder', res.id);                         // efecto
}
```

En React Native: en React, los efectos secundarios viven en `useEffect`, handlers de eventos o en capas de datos (React Query). El render debe ser puro.

Parte 3. Librerías y decisiones del ecosistema

Librerías de utilidades (Lodash) y alternativas nativas

Lodash ofrece decenas de utilidades (`debounce`, `groupBy`, `cloneDeep`, `isEqual`...). Hoy JavaScript nativo cubre muchas, así que conviene preguntarse si de verdad necesitas la dependencia.

- **Alternativas nativas:** `structuredClone` (copia profunda), `Array.prototype.flat/flatMap/at`, `Object.entries/fromEntries`, `Object.groupBy` (ES2024), optional chaining `?.` y nullish `??` (reemplazan `_.get`).
- **Cuándo sí se justifica:** `debounce`/`throttle` bien probados, `isEqual` profundo, `cloneDeep` con casos raros, o cuando el proyecto ya la usa.
- **Tamaño:** importa funciones sueltas (`import debounce from 'lodash/debounce'`) o usa `lodash-es`, para no inflar el bundle de la app.

```
// Lodash
import get from 'lodash/get';
const city = get(user, 'address.city', 'N/A');

// Nativo
const city2 = user.address?.city ?? 'N/A';
const copy = structuredClone(state);
```

Librerías de frontend

- **React:** UI declarativa basada en componentes.
- **Routing:** React Router (web); en React Native, React Navigation o Expo Router (basado en archivos).
- **Formularios:** React Hook Form (poco re-render) con resolvers de Zod; Formik es más antiguo.
- **Accesibilidad:** en web, eslint-plugin-jsx-a11y y librerías headless (Radix, React Aria); en RN, las props `accessibility*` del propio framework.
- **Clientes de API:** `fetch` nativo con un pequeño wrapper, axios o ky; **Apollo Client** para GraphQL (con caché normalizada); **TanStack Query** para cachear datos REST.

Librerías de backend

- **Frameworks HTTP:** Express (clásico y simple), Fastify (rápido, con esquemas), NestJS (estructura tipo Angular, muy usado en equipos grandes), Hono.
- **Validación:** Zod, Joi, class-validator (NestJS).
- **Logging:** pino (rápido, JSON) o winston. Logs estructurados con ids de correlación.
- **Clientes de base de datos:** `pg`, `mysql2`, drivers nativos.
- **ORMs: Prisma** (esquema declarativo, cliente tipado, migraciones), **Sequelize** (más antiguo, patrón Active Record con modelos), TypeORM, Drizzle.

Cómo evaluar una dependencia

Cada dependencia es código ajeno que mantendrás. Antes de añadirla, revisa:

- **Mantenimiento:** última publicación, issues abiertos, número de mantenedores.
- **Seguridad:** vulnerabilidades conocidas (`npm audit`), historial de CVEs.
- **Tamaño del bundle:** importante en móvil (bundlephobia, bundle visualizer).
- **Compatibilidad:** versión de React Native/Expo, New Architecture, Hermes, TypeScript.
- **Licencia:** compatible con tu producto (MIT/Apache suelen ser seguras).

- **Estabilidad de la API:** cambios incompatibles frecuentes son costo oculto.
- **Costo de migración:** ¿qué tan difícil será reemplazarla? Aísla su uso detrás de una capa propia.
- **Convenciones del proyecto:** prefiere las utilidades compartidas y bien mantenidas que ya existen antes de añadir otra que se solape (no tener dos librerías de fechas o de HTTP).

Leer documentación y escribir un wrapper

Una habilidad senior es leer la documentación buscando **casos límite**: qué pasa con errores, `null`, timeouts, cancelación, versiones. Cuando una librería es muy flexible o su API es incómoda, un **wrapper pequeño** la restringe a lo que tu proyecto necesita, centraliza la configuración y facilita cambiarla después.

```
// Wrapper minimo sobre fetch: base URL, auth, timeout y errores consistentes
export async function http<T>(path: string, init: RequestInit = {}): Promise<T> {
  const ctrl = new AbortController();
  const t = setTimeout(() => ctrl.abort(), 10_000);
  try {
    const res = await fetch(`${BASE_URL}${path}`, {
      ...init,
      signal: ctrl.signal,
      headers: { 'Content-Type': 'application/json', ...authHeader(), ...init.headers },
    });
    if (!res.ok) throw new ApiError(res.status, await res.text());
    return (await res.json()) as T;
  } finally {
    clearTimeout(t);
  }
}
```

Parte 4. APIs: HTTP, GraphQL, seguridad y trabajo asíncrono

HTTP: métodos, códigos de estado y cabeceras

HTTP es el protocolo en el que se basan casi todas las APIs que consumirá tu app. Cada petición tiene un **método**, una URL, **cabeceras** y a veces un cuerpo; la respuesta trae un **código de estado**.

Método	Uso	Seguro	Idempotente
GET	Leer un recurso	Sí	Sí
POST	Crear o ejecutar una acción	No	No
PUT	Reemplazar un recurso completo	No	Sí
PATCH	Modificar parcialmente	No	Depende
DELETE	Eliminar	No	Sí

Seguro = no cambia nada en el servidor. **Idempotente** = repetir la petición varias veces deja el mismo resultado que hacerla una vez (clave para reintentos).

Rango	Significado	Códigos frecuentes
2xx	Éxito	200 OK, 201 Created, 204 No Content
3xx	Redirección	301, 302, 304 Not Modified
4xx	Error del cliente	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable, 429 Too Many Requests
5xx	Error del servidor	500, 502, 503 Service Unavailable, 504 Gateway Timeout

- **Cabeceras clave:** `Content-Type` (formato del cuerpo), `Accept`, `Authorization` (credenciales), `Cache-Control` y `ETag` (caché), `Idempotency-Key` (evitar duplicados), `Retry-After` (con 429/503).
- **401 vs 403:** 401 = no sabemos quién eres (no autenticado); 403 = sabemos quién eres pero no tienes permiso.

Diseño de APIs REST

- **Recursos como sustantivos en plural:** `/users`, `/users/42/orders`. Las acciones las expresa el método HTTP, no la URL (`POST /users`, no `/createUser`).
- **Códigos de estado correctos** y un **formato de error consistente** (código, mensaje, detalles).
- **Paginación, filtrado y orden:** `?limit=20&cursor=abc&sort=-createdAt`. Preferible paginación por cursor en feeds.
- **Versionado:** `/v1/...` o por cabecera, para no romper apps ya instaladas (en móvil no controlas cuándo se actualizan los usuarios).
- **Idempotencia** en operaciones sensibles (pagos) con `Idempotency-Key`.

```
GET    /v1/users?limit=20&cursor=abc     -> 200 { items: [...], nextCursor: "def" }
POST   /v1/users                         -> 201 + Location: /v1/users/42
PATCH  /v1/users/42                      -> 200 usuario actualizado
DELETE /v1/users/42                      -> 204

// Error consistente
422 { "error": { "code": "VALIDATION", "message": "Email invalido",
      "fields": { "email": "formato" } } }
```

En React Native: como no puedes forzar a todos a actualizar la app, el backend debe mantener compatibilidad hacia atrás y la app debe tolerar campos nuevos o ausentes (otra razón para validar con

Zod).

GraphQL: esquemas, queries, mutations, resolvers y errores

GraphQL es un lenguaje de consulta donde **el cliente pide exactamente los campos que necesita** en un único endpoint, con un esquema fuertemente tipado.

- **Schema:** define tipos y operaciones. **Query** = leer. **Mutation** = modificar. **Subscription** = eventos en tiempo real.
- **Resolvers:** funciones del servidor que resuelven cada campo; pueden llamar a la base de datos o a otros servicios.
- **Problema N+1:** un resolver por elemento de una lista puede disparar una consulta por elemento. Se resuelve con **DataLoader** (agrupa y cachea por petición).
- **Errores:** la respuesta puede traer `data` *y* `errors` a la vez (resultados parciales), normalmente con HTTP 200. Se pueden añadir códigos en `extensions`. Para errores de negocio esperados, muchos esquemas usan **tipos unión** (`CreateUserResult = User | ValidationError`).
- **Cliente (Apollo):** caché normalizada por `id` y `__typename`; después de una mutation hay que actualizar o invalidar la caché.

```
# Schema
type User { id: ID!  name: String!  posts: [Post!]! }
type Post { id: ID!  title: String! }
type Query    { user(id: ID!): User }
type Mutation { createPost(title: String!): Post! }

# Query del cliente: solo pide lo necesario
query { user(id: "1") { name posts { title } } }
```

```
// Resolver con DataLoader (evita N+1)
const resolvers = {
  User: {
    posts: (user, _args, ctx) => ctx.loaders.postsByUserId.load(user.id),
  },
};

// Respuesta con error parcial
{ "data": { "user": null },
  "errors": [{ "message": "No autorizado", "extensions": { "code": "FORBIDDEN" } }] }
```

Autenticación vs autorización

- **Autenticación (AuthN):** ¿quién eres? Se demuestra con credenciales, tokens, biometría.
- **Autorización (AuthZ):** ¿qué puedes hacer? Roles (RBAC), atributos o pertenencia (ABAC), propiedad del recurso.
- **Se valida en el servidor, siempre.** Lo que la app oculta o bloquea es solo comodidad de UX; un atacante puede llamar a la API directamente. Esto aplica también a `isAdmin` o `userId` que vengan en el cuerpo de la petición: no se confía en ellos.
- **Riesgo clásico (IDOR):** `GET /orders/123` que devuelve el pedido de otro usuario porque solo se comprobó que estás logueado, no que sea tuyo.
- **Tokens:** JWT de vida corta + refresh token; el servidor verifica firma, expiración, audiencia y emisor. Los tokens se guardan en el almacenamiento seguro del dispositivo (Keychain/Keystore).

```
// Middleware de autenticacion
function requireAuth(req, res, next) {
  const token = req.headers.authorization?.replace('Bearer ', '');
  try { req.user = verifyJwt(token); next(); }
  catch { res.status(401).json({ error: 'No autenticado' }); }
}
```

```
// Autorizacion: comprobar propiedad del recurso
app.get('/orders/:id', requireAuth, async (req, res) => {
  const order = await orders.find(req.params.id);
  if (!order) return res.status(404).end();
  if (order.userId !== req.user.id && req.user.role !== 'admin')
    return res.status(403).json({ error: 'Sin permiso' });
  res.json(order);
});
```

Separación de transporte, lógica de negocio y persistencia

Dividir el backend en capas hace el código testeable y evita que un cambio en una parte rompa todo.

- **Transporte (controller/handler):** recibe la petición HTTP o GraphQL, valida el formato, llama al servicio y arma la respuesta. No contiene reglas de negocio.
- **Lógica de negocio (service/use case):** reglas de la aplicación, independientes de HTTP y de la base de datos.
- **Persistencia (repository/DAO):** habla con la base de datos u ORM.
- **Beneficio:** puedes probar la lógica sin levantar un servidor, y cambiar de ORM o de framework sin reescribir las reglas.

```
// controller (transporte)
app.post('/orders', requireAuth, async (req, res) => {
  const input = OrderSchema.parse(req.body);
  const order = await orderService.create(req.user.id, input);
  res.status(201).json(order);
});

// service (negocio)
class OrderService {
  constructor(private repo: OrderRepository) {}
  async create(userId: string, input: OrderInput) {
    if (input.items.length === 0) throw new DomainError('Pedido vacio');
    return this.repo.insert({ userId, ...input, total: sum(input.items) });
  }
}

// repository (persistencia)
class OrderRepository {
  insert(data: NewOrder) { return prisma.order.create({ data }); }
}
```

Trabajo asíncrono: colas, reintentos, timeouts, idempotencia y duplicados

Las tareas lentas o poco fiables (enviar un correo, procesar un pago, generar un reporte) no deben bloquear la respuesta HTTP. Se encolan y un *worker* las procesa en segundo plano.

- **Colas (queues):** BullMQ/Redis, SQS, RabbitMQ, Kafka. Desacoplan y absorben picos de carga.
- **Reintentos:** ante fallos temporales se reintenta con **backoff exponencial y jitter** (esperas crecientes con algo de aleatoriedad) para no saturar al servicio. Solo se reintentan operaciones idempotentes o protegidas.
- **Timeouts:** toda llamada externa debe tener un límite de tiempo. En el cliente se usa `AbortController`.
- **Idempotencia:** que repetir una operación no cree efectos duplicados. Se logra con una `Idempotency-Key` que el servidor recuerda.
- **Eventos duplicados:** la mayoría de sistemas de mensajería entregan *al menos una vez*, así que un mismo mensaje puede llegar dos veces. Se maneja guardando el id del evento procesado (con restricción única en la base de datos) y/o haciendo el procesamiento idempotente; el patrón **outbox** evita perder eventos al guardar y publicar.

```
async function withRetry<T>(fn: () => Promise<T>, max = 4): Promise<T> {
  for (let attempt = 0; ; attempt++) {
    try { return await fn(); }
    catch (e) {
      if (attempt >= max) throw e;
      const delay = Math.min(1000 * 2 ** attempt, 10_000) * (0.5 + Math.random());
      await new Promise(r => setTimeout(r, delay));
    }
  }
}

// Procesamiento idempotente de eventos
async function handleEvent(evt: { id: string; payload: unknown }) {
  try {
    await db.processedEvents.create({ data: { id: evt.id } });  // UNIQUE(id)
  } catch (e) {
    if (isUniqueViolation(e)) return;                           // ya procesado
    throw e;
  }
  await doWork(evt.payload);
}
```

En React Native: en el cliente, para pagos o envíos, genera una `Idempotency-Key` (UUID) por intención del usuario y reutilízala en los reintentos; así un doble toque o una reconexión no duplica la orden.

Parte 5. Bases de datos y ORMs

Ventajas y límites de un ORM

Un **ORM** (Object-Relational Mapper) te deja trabajar con la base de datos usando objetos y métodos de tu lenguaje en vez de escribir SQL a mano.

- **Ventajas:** productividad, tipado (Prisma genera tipos), migraciones, protección básica contra inyección SQL, portabilidad entre bases de datos.
- **Límites:** puede generar consultas ineficientes sin que lo notes (N+1, joins innecesarios, traer columnas de más), abstrae mal consultas complejas (reportes, ventanas, CTEs) y esconde cómo funciona la base de datos.
- **Cuándo inspeccionar el SQL generado:** endpoints lentos, listas con relaciones, consultas con muchos filtros. En Prisma se activa con `log: ['query']`.
- **Cuándo usar SQL directo:** consultas analíticas, agregaciones complejas, operaciones masivas, o cuando el ORM no expresa lo que necesitas (`$queryRaw` con parámetros, nunca concatenando strings).

```
const prisma = new PrismaClient({ log: ['query'] });   // ver el SQL real

// Consulta con ORM
const users = await prisma.user.findMany({
  where: { active: true },
  select: { id: true, name: true },                    // solo lo necesario
});

// SQL directo parametrizado cuando conviene
const rows = await prisma.$queryRaw`
  SELECT date_trunc('day', created_at) AS day, count(*) AS n
  FROM orders WHERE user_id = ${userId} GROUP BY 1 ORDER BY 1`;
```

Paginación

- **Offset/limit** (`LIMIT 20 OFFSET 400`): simple, permite saltar a una página, pero se vuelve lento con offsets grandes y puede duplicar/omitir elementos si hay inserciones mientras se pagina.
- **Cursor/keyset** (`WHERE id < :ultimoId ORDER BY id DESC LIMIT 20`): rendimiento estable con tablas grandes y resultados consistentes. Ideal para feeds con scroll infinito, aunque no permite saltar a una página arbitraria. Requiere un orden estable (por ejemplo, `createdAt` + `id`) y un índice.

```
// Keyset con Prisma
const items = await prisma.post.findMany({
  take: 20,
  skip: cursor ? 1 : 0,
  cursor: cursor ? { id: cursor } : undefined,
  orderBy: { id: 'desc' },
});
const nextCursor = items.at(-1)?.id ?? null;
```

El problema N+1

Ocurre cuando haces **1 consulta** para traer una lista y luego **N consultas más** (una por elemento) para traer sus relaciones. Con 100 posts son 101 consultas en lugar de 1 o 2.

- **Cómo detectarlo:** activar el log de consultas, ver cuántas se ejecutan por petición, usar herramientas de APM.
- **Cómo resolverlo:** cargar relaciones con `include`/`join` (eager loading), consultas por lotes con `WHERE id IN (...)`, o `DataLoader` en GraphQL.

```
// N+1: una consulta por cada post
const posts = await prisma.post.findMany();
for (const p of posts) {
  p.author = await prisma.user.findUnique({ where: { id: p.authorId } });
}

// Solucion: una sola consulta con relacion
const posts2 = await prisma.post.findMany({ include: { author: true } });
```

Bloqueos (locking) y concurrencia

Cuando dos peticiones modifican el mismo dato a la vez, puede perderse una actualización (por ejemplo, dos compras que descuentan el mismo stock).

- **Transacciones:** agrupan operaciones que deben cumplirse todas o ninguna (atomicidad).
- **Bloqueo optimista:** no bloquea; guarda una columna `version` y al actualizar comprueba que no haya cambiado (`UPDATE ... WHERE id = 1 AND version = 3`). Si falla, se reintenta. Bueno cuando los conflictos son raros.
- **Bloqueo pesimista:** bloquea la fila al leerla (`SELECT ... FOR UPDATE`) hasta terminar la transacción. Seguro cuando los conflictos son frecuentes, pero reduce concurrencia y puede causar *deadlocks*.
- **Niveles de aislamiento:** read committed, repeatable read, serializable; a más aislamiento, más garantías y menos concurrencia.

```
-- Bloqueo pesimista
BEGIN;
SELECT stock FROM products WHERE id = 7 FOR UPDATE;
UPDATE products SET stock = stock - 1 WHERE id = 7;
COMMIT;

-- Operacion atomica (a menudo la mejor opcion)
UPDATE products SET stock = stock - 1 WHERE id = 7 AND stock > 0;
```

Diagnóstico de rendimiento

- **Mide primero:** logs de consultas lentas, APM, métricas de latencia por endpoint.
- `EXPLAIN` **/** `EXPLAIN ANALYZE`**:** muestra el plan de ejecución; busca recorridos secuenciales completos (`Seq Scan`) sobre tablas grandes.
- **Índices:** aceleran lecturas en columnas usadas en `WHERE`, `JOIN` y `ORDER BY`; cuestan espacio y ralentizan escrituras. Los índices compuestos dependen del orden de las columnas.
- **Consultas más ligeras:** traer solo las columnas necesarias, evitar N+1, paginar, cachear resultados costosos.
- **Infraestructura:** connection pooling (muchas conexiones abiertas degradan la base), réplicas de lectura.

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE user_id = 42 ORDER BY created_at DESC LIMIT 20;

CREATE INDEX idx_orders_user_created ON orders (user_id, created_at DESC);
```

Parte 6. React (y React Native)

Componentes, props, estado, composición y hooks

- **Componente:** función que recibe `props` y devuelve UI. Debe ser **pura** respecto a sus entradas.
- **Props:** datos que el padre pasa al hijo (solo lectura). **Estado:** datos que el componente recuerda y que, al cambiar, provocan un nuevo render.
- **Composición:** construir UIs combinando componentes pequeños, usando `children` y props de render en vez de herencia.
- **Hooks:** `useState`, `useReducer`, `useEffect`, `useRef`, `useMemo`, `useCallback`, `useContext`, y los tuyos propios (`useAuth`, `useDebounce`) para reutilizar lógica con estado. Reglas: solo en el nivel superior del componente y solo dentro de componentes o hooks.

```
type CardProps = { title: string; children: React.ReactNode };

function Card({ title, children }: CardProps) {
  return (
    <View style={styles.card}>
      <Text style={styles.title}>{title}</Text>
      {children}                      {/* composicion */}
    </View>
  );
}

// Hook propio
function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = useCallback(() => setOn(v => !v), []);
  return [on, toggle] as const;
}
```

Renderizado, efectos, estado innecesario y closures obsoletos

- **Cuándo se renderiza un componente:** cuando cambia su estado, sus props o un contexto que consume, o cuando se re-renderiza su padre. Renderizar no es lo mismo que actualizar la pantalla: React compara y aplica solo las diferencias.
- `useEffect` sincroniza el componente con algo *externo* (red, suscripciones, temporizadores). Tiene **array de dependencias** (cuándo se re-ejecuta) y una **función de limpieza** (cancelar suscripciones, timers y peticiones).
- **Evitar estado innecesario:** si un valor se puede *calcular* a partir de props o estado, no lo guardes en otro estado (estado derivado duplicado se desincroniza). No uses `useEffect` para transformar datos que puedes calcular durante el render.
- **Closure obsoleto (stale closure):** una función que 'captura' valores antiguos del render en que se creó. Suele pasar con `setInterval`, listeners y dependencias omitidas. Soluciones: actualización funcional (`setX(prev => ...)`), declarar bien las dependencias, o guardar el valor más reciente en una `ref`.

```
// Bug: stale closure (count siempre vale 0 dentro del intervalo)
useEffect(() => {
  const id = setInterval(() => setCount(count + 1), 1000);
  return () => clearInterval(id);
}, []);                                  // dependencias incompletas

// Solucion: actualizacion funcional
useEffect(() => {
  const id = setInterval(() => setCount(c => c + 1), 1000);
  return () => clearInterval(id);
}, []);
```

```
// Estado derivado: calcular, no almacenar
const [items, setItems] = useState<Item[]>([]);
const total = items.reduce((s, i) => s + i.price, 0);   // sin useState extra

// Efecto con cancelacion de peticion
useEffect(() => {
  const ctrl = new AbortController();
  fetch(url, { signal: ctrl.signal }).then(r => r.json()).then(setData).catch(() => {});
  return () => ctrl.abort();
}, [url]);
```

Para la entrevista: 'un efecto es para sincronizar con sistemas externos; si puedo calcularlo en el render, no necesito un efecto ni un estado extra'.

Formularios: estado, validación y retroalimentación

- **Controlado vs no controlado:** controlado = el valor vive en el estado de React (`value` + `onChangeText`); no controlado = el valor se lee al enviar. React Hook Form usa un enfoque mixto con pocos re-renders.
- **Validación:** en el cliente para UX rápida (Zod + React Hook Form) y **siempre también en el servidor**. Mostrar errores por campo, cuando el usuario ya interactuó (blur o submit) y no desde la primera tecla.
- **Estados de la pantalla:** carga (deshabilitar el botón, mostrar spinner, evitar doble envío), error (mensaje claro y opción de reintentar), vacío (explicar qué hacer) y éxito (confirmación).
- **Retroalimentación al usuario:** mensajes claros, toasts o banners, foco en el primer campo con error, y accesibilidad de los mensajes.

```
const schema = z.object({
  email: z.string().email('Correo invalido'),
  password: z.string().min(8, 'Minimo 8 caracteres'),
});
type FormData = z.infer<typeof schema>;

const { control, handleSubmit, formState: { errors, isSubmitting } } =
  useForm<FormData>({ resolver: zodResolver(schema) });

<Controller control={control} name="email" render={({ field }) => (
  <TextInput value={field.value} onChangeText={field.onChange}
             keyboardType="email-address" autoCapitalize="none" />
)} />
{errors.email && <Text accessibilityRole="alert">{errors.email.message}</Text>}
<Button title="Entrar" disabled={isSubmitting} onPress={handleSubmit(onSubmit)} />
```

En React Native: cuida el teclado (`KeyboardAvoidingView`, `keyboardShouldPersistTaps`), los tipos de teclado, el autocompletado de contraseñas (`textContentType`, `autoComplete`) y el orden de enfoque entre campos.

Rutas, vistas protegidas, obtención de datos, caché e invalidación

- **Rutas:** en web, React Router; en RN, React Navigation o Expo Router. Define stacks, tabs y deep links.
- **Vistas protegidas:** mostrar un stack u otro según la sesión (por ejemplo, un `Auth` stack si no hay usuario y el `App` stack si lo hay). Es solo UX; la protección real está en el servidor.
- **Obtención de datos:** usa una librería de estado de servidor (TanStack Query, Apollo) en vez de `useEffect` + `useState` manuales: da caché, deduplicación, reintentos y estados de carga/error.
- **Caché e invalidación:** cada consulta tiene una clave; tras una mutación, invalidas las claves afectadas para refrescar, o actualizas la caché de forma optimista y haces rollback si falla.

```
// Navegacion segun sesion
function RootNavigator() {
```

```
  const { user, loading } = useAuth();
  if (loading) return <Splash />;
  return user ? <AppStack /> : <AuthStack />;
}

// Datos con cache e invalidacion
const qc = useQueryClient();
const { data, isPending, isError } = useQuery({
  queryKey: ['todos'], queryFn: api.getTodos,
});
const addTodo = useMutation({
  mutationFn: api.createTodo,
  onSuccess: () => qc.invalidateQueries({ queryKey: ['todos'] }),
});
```

Accesibilidad, diseño responsivo y renderizado seguro de contenido del usuario

- **Accesibilidad (a11y):** que personas con lector de pantalla, baja visión o motricidad reducida puedan usar la app. En React Native: `accessibilityLabel`, `accessibilityRole`, `accessibilityState`, `accessibilityHint`, `accessible`; tamaños táctiles mínimos (44x44 pt en iOS, 48 dp en Android), contraste suficiente, soporte de tamaño de fuente del sistema, y no transmitir información solo con color. Prueba con VoiceOver (iOS) y TalkBack (Android).
- **Diseño responsivo:** usa flexbox, porcentajes y `useWindowDimensions` en vez de tamaños fijos; considera tabletas, orientación horizontal, áreas seguras (`SafeAreaView`) y pantallas pequeñas. En web, media queries y unidades relativas.
- **Renderizado seguro de contenido de usuario:** nunca confíes en texto ajeno. En web, React escapa el contenido por defecto; evita `dangerouslySetInnerHTML` (si es imprescindible, sanitiza con DOMPurify). En React Native, `<Text>` renderiza texto plano de forma segura; el riesgo aparece con `WebView` o al mostrar HTML/Markdown de terceros: sanitiza, restringe `originWhitelist`, desactiva JavaScript si no hace falta y valida los enlaces antes de abrirlos (`Linking`).

```
<Pressable
  onPress={onLike}
  accessibilityRole="button"
  accessibilityLabel="Me gusta"
  accessibilityState={{ selected: liked }}
  style={{ minHeight: 48, minWidth: 48 }}
>
  <Icon name={liked ? 'heart' : 'heart-outline'} />
</Pressable>

const { width } = useWindowDimensions();
const columns = width > 700 ? 3 : 1;          // responsivo
```

Lista de autoevaluación

Marca mentalmente cada punto. Si no puedes explicarlo en dos frases con un ejemplo, vuelve a ese tema.

- Explico la diferencia entre `interface` y `type`, y qué es el declaration merging.
- Sé cuándo usar un enum, una unión de literales o un objeto `as const`.
- Distingo inferencia, `as` y `satisfies`, y sé por qué `as` no valida en runtime.
- Escribo una función genérica con constraint y una sobrecarga.
- Uso uniones discriminadas con comprobación exhaustiva (`never`).
- Conozco `Partial`, `Pick`, `Omit`, `Record`, `ReturnType`, `Parameters` y sé cuándo evitar tipos complejos.
- Configuro `strict` y sé migrar un proyecto JS a TS de forma incremental.
- Valido datos externos con Zod y explico por qué los tipos no bastan.
- Elijo `Map`, `Set`, pila, cola, árbol o grafo según el patrón de acceso y calculo su complejidad.
- Explico funciones puras, inmutabilidad, composición y efectos en los límites.
- Evalúo una dependencia (mantenimiento, seguridad, tamaño, licencia, migración).
- Diseño una API REST (métodos, códigos, cabeceras, paginación, versionado) y entiendo GraphQL (schema, resolvers, errores, N+1).
- Separo autenticación de autorización y valido permisos en el servidor.
- Describo controller / service / repository.
- Manejo colas, reintentos con backoff, timeouts, idempotencia y eventos duplicados.
- Sé cuándo un ORM ayuda o estorba, detecto N+1 y elijo paginación por cursor.
- Explico bloqueo optimista vs pesimista y cómo diagnosticar una consulta lenta.
- En React: hooks, efectos con limpieza, stale closures, estado derivado, formularios, caché/invalidación y accesibilidad.

Cómo practicar

- Resuelve retos de TypeScript en Type Challenges y ejercicios de la pista TS de Exercism.
- Construye una mini app en Expo con: lista paginada (React Query), formulario con Zod, estados de carga/error/vacío y navegación con rutas protegidas.
- Monta una API pequeña (Express o Fastify + Prisma) con auth JWT y paginación por cursor; actívale el log de consultas y busca un N+1 a propósito para corregirlo.
- Explica en voz alta cada tema como si se lo enseñaras a alguien: es la mejor prueba de que lo entiendes.